



LIMBUS SPLIT PRO
DOCUMENTO DE CONSTRUCCIÓN

Prompt maestro para Codex

Diseño, motor de separación, reproducción,
licencias, pruebas y empaquetado de macOS.

VERSIÓN 1.0 · JULIO 2026

LIMBUS SPLIT PRO

Prompt maestro para construir la aplicación desde cero con Codex

Versión 1.0 - Julio de 2026

Cómo utilizar este documento

1. Crea una tarea nueva en Codex dentro de una carpeta de proyecto vacía.
2. Copia el prompt completo incluido a continuación.
3. Adjunta cualquier referencia visual o archivo de audio de prueba que quieras utilizar.
4. Deja que Codex inspeccione el entorno, investigue modelos, implemente, pruebe y empaquete la aplicación.
5. Revisa siempre las licencias de los modelos y dependencias antes de distribuir el resultado.

IMPORTANTE: Este documento contiene instrucciones de desarrollo. No incluye modelos entrenados ni concede derechos sobre software, datasets o pesos de terceros. Cada usuario es responsable de comprobar las licencias de los componentes que incorpore.

PROMPT COMPLETO - COPIAR DESDE AQUÍ

Actúa como arquitecto de software, desarrollador senior de macOS, especialista en audio digital, machine learning aplicado a separación musical, diseño de producto y empaquetado de aplicaciones.

Quiero que diseñes, programes, pruebes y empaquetes desde cero una aplicación de escritorio para macOS llamada Limbus Split Pro.

Trabaja de forma autónoma hasta obtener una aplicación funcional y verificable. No te limites a describir código: inspecciona el entorno, investiga, implementa, compila, ejecuta pruebas reales y genera los artefactos finales.

Si encuentras un impedimento técnico o legal, no lo ocultes ni simules que una función existe. Continúa con todo lo que sí pueda completarse y explica exactamente qué modelo, licencia, certificado o recurso falta.

1. Objetivo general

Limbus Split Pro será una aplicación nativa para separar una mezcla musical en pistas independientes mediante modelos locales de inteligencia artificial.

Debe estar pensada para músicos, productores, alumnos y usuarios sin conocimientos técnicos. El flujo debe ser directo: cargar una canción, elegir qué separar, escoger una carpeta, procesar, escuchar las pistas y exportar.

Prioridades, en este orden:

1. Calidad real de separación.
2. Sincronización exacta de reproducción y exportación.
3. Estabilidad y ausencia de archivos silenciosos falsos.
4. Facilidad de uso.
5. Privacidad y procesamiento local.
6. Portabilidad del instalador.
7. Licencias y atribuciones correctas.
8. Diseño minimalista, bonito y trabajado.

No construyas una página web ni una aplicación Electron. Desarrolla una aplicación nativa de macOS, preferiblemente con Swift, SwiftUI, AppKit, AVFoundation y AVAudioEngine.

2. Nombre, identidad y diseño

Nombre del producto: Limbus Split Pro.

El diseño debe ser:

- Minimalista.
- Limpio.
- Elegante.
- Profesional.
- Amigable.
- Con personalidad visual.

- Con una paleta de color reconocible y algo divertida, sin resultar infantil.

Evita una interfaz genérica, excesivamente gris o con aspecto de prototipo.

No utilices el eslogan “Una mezcla. Solo lo que necesitas”.

Respetar el área superior de las ventanas de macOS. Los controles para cerrar, minimizar y ampliar nunca deben solaparse con el logotipo, el nombre de la aplicación ni otros textos.

La interfaz debe adaptarse correctamente al tamaño de ventana y seguir siendo legible en pantallas pequeñas. Utiliza una jerarquía tipográfica clara, espacios generosos, bordes suaves y estados visuales comprensibles.

3. Flujo principal

El usuario debe poder:

1. Arrastrar una canción o seleccionarla mediante un diálogo nativo.
2. Ver su nombre, formato, duración, frecuencia de muestreo y canales.
3. Elegir exactamente qué instrumentos desea separar.
4. Elegir una carpeta de trabajo y exportación.
5. Iniciar la separación.
6. Ver el progreso real del proceso.
7. Cancelar de forma segura.
8. Escuchar simultáneamente todas las pistas generadas.
9. Aplicar mute, solo y volumen independiente.
10. Cambiar el punto de reproducción mediante una línea temporal.
11. Exportar pistas individuales.
12. Exportar una mezcla nueva con los volúmenes, mute y solo actuales.
13. Abrir la carpeta de resultados.

Nunca guardes resultados en una ubicación desconocida. El usuario debe elegir explícitamente la carpeta antes de separar.

4. Selector de pistas

La interfaz debe disponer de controles “Todas” y “Ninguna”, además de selectores individuales para cada categoría disponible.

Categorías deseadas:

- Voces.
- Voz principal.
- Coros y segundas voces.
- Efectos vocales, reverberación o ambiente vocal.
- Ruido o artefactos.
- Batería completa.

- Bombo.
- Caja.
- Toms.
- Platos.
- Bajo.
- Guitarra.
- Guitarra acústica, solo si existe un modelo realmente capaz.
- Guitarra eléctrica, solo si existe un modelo realmente capaz.
- Piano y teclados.
- Otros instrumentos que puedan separarse mediante modelos reales y verificables.
- Other.

No muestres opciones falsas ni generes archivos silenciosos para aparentar que una separación existe.

Si no existe un modelo técnicamente válido y con licencia adecuada para una categoría:

- Deja la opción desactivada.
- Explica por qué no está disponible.
- No generes una pista vacía.
- Registra la limitación en el informe.

5. Lógica de Other

La lógica de complemento es esencial.

- Si el usuario selecciona solamente “Voces”, exporta “Voces.wav” y “Other.wav”.
- En ese caso, Other debe contener todo lo que no sea voz.
- Si selecciona solamente “Batería”, exporta “Batería completa.wav”, sus detalles cuando corresponda, y “Other.wav”.
- En ese caso, Other debe contener todo lo que no sea batería.
- Si selecciona varios instrumentos, Other debe contener todo aquello que no haya sido seleccionado.
- Si selecciona todas las categorías disponibles, genera Other solamente si queda información residual que no pertenezca de forma fiable a ninguna pista.
- Other no debe ser un archivo silencioso creado artificialmente.
- La suma de las pistas principales y Other debe reconstruir la mezcla con el menor error posible.
- Evita duplicar material cuando se exporta simultáneamente una pista general y sus subpistas.

Para voces detalladas:

- La voz principal y los coros deben proceder de una separación vocal real.
- Los efectos vocales no deben ser una copia de la voz.
- El ruido debe ser una pista independiente real.
- Si una separación depende de obtener primero la voz completa, ejecuta internamente esa etapa aunque el usuario no haya marcado “Voces”.

- No obligues al usuario a entender dependencias internas entre modelos.

6. Investigación y motor de separación

Investiga modelos actuales antes de escogerlos. Utiliza fuentes primarias: repositorios oficiales, model cards oficiales, publicaciones de los autores y textos de licencia incluidos con los pesos.

Condiciones obligatorias:

- Procesamiento completamente local.
- Sin llamadas a APIs de pago durante la separación.
- Sin subir canciones.
- Sin descargar modelos silenciosamente mientras se procesa una pista.
- Gestión explícita de la instalación de modelos.
- Hash SHA-256 para cada archivo de modelo.
- Registro de procedencia, versión, licencia, autor y capacidades.
- Diferenciación entre licencia del código y licencia de los pesos entrenados.
- Prohibición de asumir que los pesos son comerciales porque el repositorio del código sea MIT.

Utiliza en una compilación distribuible únicamente modelos cuyos pesos tengan una licencia inequívoca para:

- Uso.
- Inclusión en aplicaciones.
- Redistribución.
- Uso comercial, cuando corresponda.
- Modificación, si el proceso la requiere.

Si la licencia de los pesos es ambigua, no los incluyas en una compilación pública o comercial.

Implementa un manifiesto de modelos con:

- Identificador.
- Ruta.
- Hash SHA-256.
- Versión.
- Origen.
- Licencia.
- Enlace a evidencia de licencia.
- Autor y atribución.
- Capacidades.
- Autorización de redistribución.
- Autorización comercial.

Implementa un verificador fail-closed. La compilación pública debe fallar si encuentra:

- Un modelo no registrado.

- Un modelo con hash alterado.
- Un modelo sin procedencia.
- Un modelo sin autorización clara.
- Una capacidad obligatoria sin modelo aprobado.

Puede existir una compilación experimental, pero debe estar marcada claramente como:

“COMPILACIÓN LOCAL DE DESARROLLO - NO DISTRIBUIR”.

No reduzcas la calidad de separación únicamente para resolver un problema de licencias. Si no existe un sustituto equivalente, informa del bloqueo y deja preparada la arquitectura para conectar posteriormente un modelo licenciado.

7. Runtime de Python

La aplicación no debe depender del Python instalado por el usuario, Homebrew, Conda, pyenv ni herramientas de desarrollo de Apple.

Incorpora un Python propio, reproducible y relocatable.

Requisitos:

- Versión exacta fijada.
- Fuente oficial.
- URL y SHA-256 de la fuente.
- Arquitectura Apple Silicon arm64.
- Objetivo mínimo de macOS definido.
- Ausencia de rutas absolutas del ordenador de compilación.
- Ausencia de enlaces a /opt/homebrew, /usr/local o /Users/nombre.
- OpenSSL integrado de manera reproducible.
- Soporte lzma integrado sin Homebrew.
- Registro de procedencia en PROVENANCE.json.
- PYTHONHOME configurado por los lanzadores.
- PYTHONNOUSERSITE=1.
- Cachés de Python y JIT fuera del bundle firmado.
- Prohibición de generar pyc, nbi o nbc dentro de la aplicación después de firmarla.

8. Reducción de dependencias

Reduce dependencias sin afectar la separación ni la calidad.

Elimina herramientas utilizadas únicamente durante la construcción, por ejemplo:

- pip.
- setuptools.
- Cython.

- Suites de prueba.
- Ejemplos.
- Compiladores.
- Backends no utilizados.
- Binarios de arquitecturas incompatibles.

No elimines una dependencia sin verificar después el funcionamiento real.

Rechaza:

- Dependencias no comerciales en una versión distribuible.
- Componentes con licencia incompatible.
- Binarios únicamente x86_64.
- Enlaces a rutas del ordenador del desarrollador.
- Modelos procedentes de mirrors sin procedencia verificable.

9. LGPL, atribuciones y SBOM

Si una biblioteca LGPL es necesaria:

- Utiliza enlace dinámico cuando corresponda.
- Incluye la licencia completa.
- Incluye el código fuente exacto o una oferta válida cuando sea obligatorio.
- Proporciona instrucciones para reemplazar la biblioteca.
- Documenta cómo volver a firmar la aplicación.
- Registra su hash y versión.

Genera automáticamente:

- THIRD_PARTY_NOTICES.txt.
- SBOM CycloneDX JSON.
- Carpeta con textos completos de licencias.
- Manifiesto de modelos.
- Informe de cumplimiento.
- Instrucciones LGPL.

El SBOM y los avisos deben generarse desde el contenido real y ya saneado de la aplicación, no desde una lista manual que pueda quedar obsoleta.

10. Arquitectura de reproducción

La reproducción debe comportarse como un mezclador multipista sencillo.

No utilices un reproductor independiente y sin coordinación para cada archivo. Utiliza un único reloj de audio y una única línea temporal, preferiblemente mediante AVAudioEngine, AVAudioPlayerNode o una arquitectura equivalente.

Todas las pistas deben:

- Comenzar exactamente en el mismo sample.
- Conservar la misma duración.
- Conservar la misma frecuencia de muestreo.
- Mantenerse sincronizadas durante toda la reproducción.
- Responder juntas a play, pausa, stop y seek.

Controles generales:

- Play.
- Pausa.
- Stop, si su comportamiento está claramente diferenciado.
- Barra de progreso.
- Tiempo actual.
- Duración total.
- Clic y arrastre sobre la barra para cambiar el punto de reproducción.
- Tecla espacio para alternar play y pausa.

El atajo de espacio no debe activarse mientras el usuario escribe en un campo de texto.

11. Comportamiento en tiempo real

- Pulsar mute no debe detener la reproducción.
- Pulsar solo no debe detener la reproducción.
- Cambiar el volumen no debe detener la reproducción.
- Ninguna de estas operaciones debe devolver el cursor al inicio.
- Hacer seek durante la reproducción debe continuar desde el nuevo punto.
- El seek debe ser suave, sin rebote, doble ataque, clics ni silencios perceptibles.
- Implementa un crossfade corto o rampas de ganancia al hacer seek.
- No reinicies todo el motor para cambiar mute, solo o volumen.
- Play después de pausa debe continuar desde la posición actual.
- El significado de Stop y “volver al inicio” debe ser explícito.

12. Mezclador

Cada pista debe mostrar:

- Nombre.
- Icono o color.
- Botón Mute.
- Botón Solo.
- Control de volumen.

- Porcentaje o valor de volumen.
- Medidor sencillo opcional, solo si funciona con fluidez.

La lógica de solo debe permitir uno o varios solos.

Cuando exista al menos un solo activo, solamente deben sonar las pistas en solo, excepto aquellas que además estén en mute.

Los cambios de ganancia deben utilizar rampas cortas para evitar clics.

13. Exportación de mezcla

Añade un botón “Exportar mezcla”.

La mezcla exportada debe respetar exactamente:

- Volumen de cada pista.
- Mute.
- Solo.
- Alineación.
- Duración.
- Frecuencia de muestreo.
- Número de canales.
- Punto inicial común.

La exportación no debe depender de la reproducción en tiempo real. Realízala offline con precisión de samples.

Evita clipping:

- Detecta el pico final.
- Avisa o aplica protección solamente cuando sea necesario.
- No normalices silenciosamente si eso cambia la intención del usuario.
- Documenta cualquier limitador o reducción de ganancia.

Exporta como mínimo WAV PCM, conservando la máxima calidad razonable.

14. Estructura de la interfaz

Panel izquierdo:

1. Archivo de entrada.
2. Qué extraer.
3. Carpeta de trabajo.
4. Botón “Separar y exportar”.

Panel derecho:

1. Estado del proceso.
2. Reproductor general.

3. Barra temporal.
4. Lista de pistas.
5. Controles Mute, Solo y Volumen.
6. Botón "Exportar mezcla".
7. Botón "Abrir carpeta".

Durante la separación muestra:

- Etapa actual.
- Progreso real.
- Modelo utilizado.
- Errores comprensibles.
- Opción de cancelación segura.

No muestres una traza interna de Python como único mensaje para el usuario. Guarda el detalle técnico en un log y muestra una explicación clara con una posible solución.

15. Seguridad de archivos

Nunca escribas modelos, cachés o temporales dentro de:

/Applications/Limbus Split Pro.app

Una aplicación instalada debe considerarse de solo lectura.

Guarda datos mutables en ubicaciones apropiadas:

- Application Support.
- Caches.
- Temporary Directory.
- Carpeta de trabajo elegida por el usuario.

No intentes crear archivos temporales dentro de Resources/Engine o de las carpetas de modelos.

Comprueba los permisos después de instalar mediante PKG, no solamente desde la carpeta de desarrollo.

Los modelos instalados deben ser legibles por usuarios normales.

16. Empaquetado de macOS

Genera:

- Limbus Split Pro.app.
- Archivo ZIP.
- Instalador PKG.

La aplicación debe instalarse en:

/Applications/Limbus Split Pro.app

El PKG debe:

- Conservar permisos correctos.
- Instalar recursos legibles.
- Permitir ejecutar el runtime.
- No requerir Homebrew.
- No requerir Python externo.
- No escribir dentro de la aplicación durante su uso.

Añade soporte para:

- Firma Developer ID Application.
- Firma Developer ID Installer.
- Hardened Runtime.
- Entitlements mínimos.
- Notarización.
- Stapling.

Si no hay certificados disponibles, genera una compilación local ad-hoc e indica claramente que no está firmada ni notarizada y que todavía no es un instalador comercial final.

17. Pruebas obligatorias

No consideres terminada la aplicación porque compile. Realiza pruebas con archivos de audio reales.

Comprueba:

1. Importación completa del runtime.
2. Python, OpenSSL y Izma.
3. PyTorch.
4. ONNX Runtime.
5. TorchAudio y TorchVision, si se distribuyen.
6. Librosa.
7. Motor principal de separación.
8. Modelos vocales.
9. Separación de ruido.
10. Separación de efectos.
11. Separación detallada de batería.
12. Duración de todas las pistas.
13. Frecuencia de muestreo.
14. Número de canales.
15. Archivos no silenciosos cuando exista señal correspondiente.
16. Sincronización sample a sample.
17. Reconstrucción de la mezcla.
18. Mute durante reproducción.

19. Solo durante reproducción.
20. Cambios de volumen durante reproducción.
21. Seek suave.
22. Barra temporal.
23. Atajo de espacio.
24. Exportación de mezcla.
25. Instalación limpia mediante PKG.
26. Ejecución desde /Applications.
27. Ejecución con la aplicación en modo de solo lectura.
28. Ausencia de rutas de Homebrew.
29. Ausencia de rutas /Users del ordenador de compilación.
30. Arquitectura arm64 de todos los binarios.
31. Firma del bundle.
32. Permisos del BOM del PKG.
33. Validez del SBOM.
34. Hashes de modelos.
35. Ausencia de modelos no autorizados en una compilación pública.

No consideres un archivo silencioso como prueba correcta de separación.

Utiliza una canción de prueba que contenga realmente el instrumento que se está validando. Para batería detallada, utiliza una grabación con bombo, caja, toms y platos reales.

18. Informes

Después de cada revisión importante entrega un informe con:

- Funciones implementadas.
- Modelos utilizados.
- Licencias verificadas.
- Dependencias eliminadas.
- Pruebas realizadas.
- Resultados.
- Errores encontrados.
- Errores corregidos.
- Limitaciones pendientes.
- Estado comercial.
- Estado de firma y notarización.
- Hashes de instaladores.

19. Entrega en cada iteración

Después de terminar cada petición, proporciona enlaces clicables a:

- Aplicación .app.
- Archivo ZIP.
- Instalador PKG.
- Informe correspondiente.

No presentes como “lista para vender” una compilación que:

- Utilice pesos con licencia ambigua.
- No esté firmada.
- No esté notarizada.
- No haya superado una instalación limpia.
- Escriba dentro de su propio bundle.
- Dependa del ordenador del desarrollador.

20. Forma de trabajo

Antes de cambiar archivos:

- Inspecciona el estado actual.
- Conserva modificaciones existentes.
- Evita operaciones destructivas.
- Documenta decisiones técnicas importantes.

Toma decisiones razonables que permitan avanzar, siempre dentro del alcance del proyecto.

Cuando una función dependa de información actual, como versiones, modelos, licencias o requisitos de notarización, verifica fuentes oficiales en lugar de confiar únicamente en memoria.

21. Criterio final de éxito

Limbus Split Pro estará terminado cuando:

- Sea una aplicación nativa y visualmente cuidada.
- Separe únicamente pistas que realmente pueda generar.
- Las pistas contengan audio real.
- Other funcione como complemento correcto.
- El mezclador sea sample-accurate.
- Mute, solo, volumen y seek funcionen sin detener la reproducción.
- La mezcla exportada coincida con lo escuchado.
- Funcione desde /Applications.
- No dependa de Homebrew ni de Python externo.

- Incorpore un runtime reproducible.
- Incluya atribuciones, SBOM y cumplimiento LGPL.
- Distribuya únicamente modelos con licencia inequívoca.
- Esté firmada, notarizada y probada mediante una instalación limpia antes de considerarla comercial.

22. Primera ejecución

Empieza en este orden:

1. Inspecciona el workspace.
2. Crea un inventario técnico.
3. Investiga modelos y prepara una tabla de licencias.
4. Diseña la arquitectura.
5. Presenta un plan de implementación.
6. Construye la primera versión funcional.
7. Ejecuta pruebas reales.
8. Corrige los fallos.
9. Genera la aplicación y los instaladores.
10. Entrega un informe verificable.

Continúa trabajando hasta entregar el mejor resultado posible, sin simular capacidades inexistentes ni ocultar limitaciones legales o técnicas.

FIN DEL PROMPT

Aviso de distribución

Este prompt puede publicarse gratuitamente para que cada usuario lo utilice en su propia cuenta de Codex.

No compartas claves API, credenciales ni cuentas. El código, las dependencias y los modelos que se incorporen durante la construcción pueden estar sujetos a licencias de terceros y deben revisarse antes de distribuir una aplicación resultante.

Codex es un producto de OpenAI. Este documento es independiente y no implica afiliación, patrocinio ni respaldo oficial de OpenAI.